

Topic Proposal for Presentation

Yucheng Cheng

Changze Wu

1. Names and Email Addresses

- Yucheng Cheng – `yucheng.cheng@mail.utoronto.ca`
- Changze Wu – `changze.wu@mail.utoronto.ca`

2. Problem Description

Problem Name: Fox And Dinner

Source: Codeforces Round #290 (Div. 2) – Problem E

Link: <https://codeforces.com/contest/510/problem/E>

Short Description: Given n foxes with specific ages, we must seat them around round tables such that:

- (1) Each fox sits at exactly one table.
- (2) Each table has at least 3 foxes.
- (3) The sum of ages of any two adjacent foxes at a table must be a prime number.

We must determine whether such a distribution is possible, and if so, output one valid configuration.

3. Algorithm Description

We will solve this problem using a **maximum flow algorithm**, specifically the **Ford-Fulkerson algorithm** as taught in lecture.

Key Observations

- A valid adjacency (edge) can only occur between an even-aged and an odd-aged fox whose age sum is a prime.
- This naturally forms a bipartite graph: one side is even-aged foxes, the other is odd-aged foxes.
- Each odd-aged fox must be connected to two even-aged foxes and vice versa, forming cycles that represent tables.

Network Flow Construction

- Create a flow network:
 - Add a source node s and a sink node t .
 - Connect each odd-aged fox to the source with capacity 2.
 - Connect each even-aged fox to the sink with capacity 2.
 - For every (odd, even) pair whose age sum is prime, add a directed edge with capacity 1 (for primality test, use trial division, which checks if any of the numbers between 2 and $\sqrt{n}$ is a divisor of n).
- Run the **Ford-Fulkerson algorithm** to compute the max flow.

Algorithm Rationale

If the maximum flow equals two times the number of even-aged foxes, a valid pairing exists. We then use these matches to reconstruct cycles (i.e., tables) where each node connects to exactly two others.

Why Ford-Fulkerson: This algorithm, taught in our lectures, iteratively finds augmenting paths in the residual graph and adjusts flows until no more augmenting paths exist. It is well-suited for small to medium-sized graphs (like this one, $n \leq 200$).

4. References and Materials

- **Problem Statement:** <https://codeforces.com/contest/510/problem/E>
- **Solution Explanation:** <https://codeforces.com/blog/entry/16173> (section “510E - Fox And Dinner / 512C - Fox And Dinner”)
- **Ford-Fulkerson Algorithm:** Slide for lecture 8 of this course.
- **Primality test:** https://en.wikipedia.org/wiki/Primality_test#Heuristic_tests (section “Simple methods”)